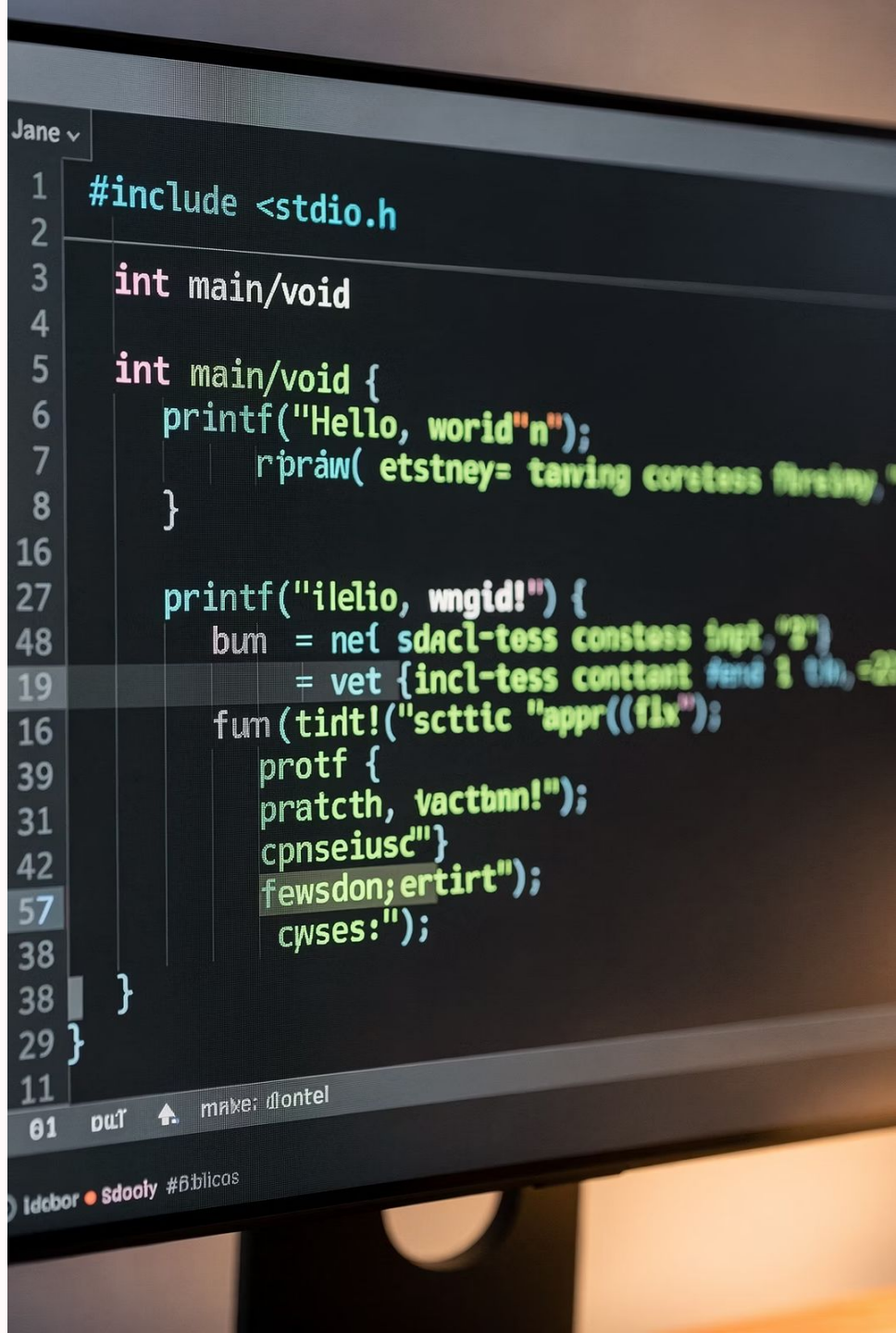
Introduzione al linguaggio C

Hello world, variabili e tipi, input/output, operatori

CORSO BASE DI PROGRAMMAZIONE



Introduzione al linguaggio C

Sottotitolo: Hello world, variabili e tipi, input/output, operatori

Hello World

Struttura minima, compilazione e primo programma funzionante

Variabili e Tipi

Interi, reali, char, costanti e booleani

Input/Output

printf e scanf con tutti gli specificatori di formato

Operatori

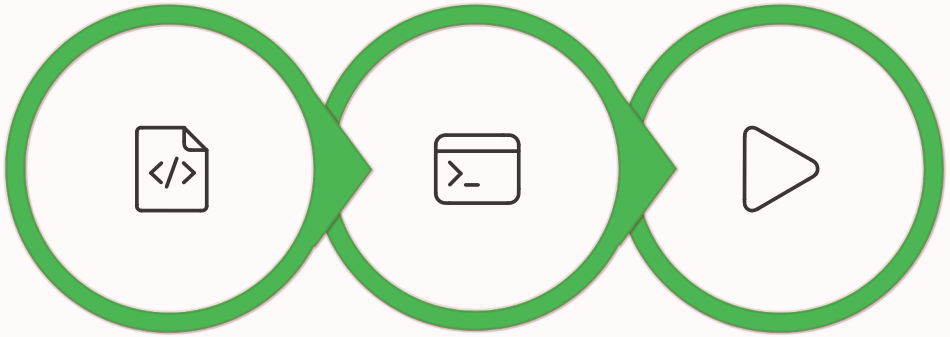
Aritmetici, relazionali, logici, assegnazione e bit a bit



Cos'è il linguaggio C

Creato nei **Bell Labs negli anni '70**, il C è un linguaggio **compilato**, **tipizzato staticamente** e **procedurale**. È ancora oggi alla base di sistemi operativi, firmware e linguaggi moderni. In questo corso si utilizza lo standard **C11**.

Il ciclo di sviluppo trasforma il sorgente in un eseguibile:



Sorgente: hello.c	**Compilatore: gcc**	**Eseguibile: hello**
file di testo scritto dal programmatore	traduce in linguaggio macchina	programma pronto da eseguire

Il comando `gcc -Wall -Wextra -o hello hello.c` attiva tutti gli avvisi utili per trovare errori prima che diventino problemi a runtime.

Struttura minima di un programma C

Ogni programma C valido ha esattamente **una funzione** `main`. Ecco le parti fondamentali, ognuna con il suo ruolo preciso:

```
#include <stdio.h>   ← direttiva al preprocessore
int main(void)       ← punto di inizio obbligatorio
{                    ← apertura del blocco
    printf("Ciao!\n");← istruzione (termina con ;)
    return 0;        ← valore di ritorno al sistema
}                    ← chiusura del blocco
```

#include <stdio.h>

Rende disponibili `printf` e `scanf`. Senza questa direttiva il compilatore non conosce le funzioni di I/O.

int main(void)

Il punto di ingresso obbligatorio. Il programma parte sempre da qui, non importa quante altre funzioni siano definite.

return 0;

Comunica al sistema operativo che il programma è terminato **senza errori**. Un valore diverso da zero segnala un problema.

Commenti nel codice C

Dal **C99** sono disponibili due sintassi di commento. Il commento spiega il **perché**, non il cosa — il codice lo dice già da solo.

```
/* Commento su più righe:
   inizia con /* e termina con */
// Commento a riga singola: dalla doppia barra alla fine della riga
```

❌ Commento inutile

```
// incrementa x di 1
x = x + 1;
```

Ripete quello che il codice già dice chiaramente.

✅ Commento utile

```
// prossima posizione libera
// nell'array
x = x + 1;
```

Spiega il *perché* di quella operazione nel contesto.

⚠️ Un commento **sbagliato è peggio di nessun commento**: induce chi legge in errore. Tieni sempre i commenti aggiornati quando modifichi il codice.

Tipi interi

I tipi interi in C variano per **dimensione in byte** e per l'**intervallo di valori** rappresentabili. Su sistemi a 64 bit:

Tipo	Byte	Minimo	Massimo
`short`	2	-32 768	32 767
`int`	4	-2 147 483 648	2 147 483 647
`long`	8	-9.2×10^{18}	9.2×10^{18}
`long long`	8	-9.2×10^{18}	9.2×10^{18}
`unsigned int`	4	0	4 294 967 295

sizeof(tipo)

Restituisce i byte occupati. I limiti precisi sono in `<limits.h>`.

Suffissi

`L` = long, `U` = unsigned, `LL` = long long. Es.: `1000000L`

Overflow

Superare il massimo "ricomincia" dall'estremo opposto. Scegli sempre il tipo adatto!

Tipi reali

I tipi reali differiscono per **precisione** (numero di cifre significative affidabili) e per la dimensione in memoria. Il confronto su π con 20 decimali è molto illustrativo:

Tipo	Byte	Cifre	π stampato con 20 decimali
`float`	4	6	`3.14159274101257324219`
`double`	8	15	`3.14159265358979323846`
`long double`	16	18	`3.14159265358979323851`

Le cifre in blu non sono affidabili — sono "rumore" di arrotondamento.

Notazione scientifica

`1.496e8` = 150 000 000 (distanza Terra-Sole in km) | `9.1e-31` = massa dell'elettrone in kg

⚠️ Non usare == tra reali

Gli errori di arrotondamento possono rendere diversi due valori matematicamente uguali. Usa una **tolleranza**: `fabs(a - b) < 1e-9`

Tipo `char` e simulazione del booleano

In C, `char` è semplicemente un **intero a 8 bit**: memorizza il codice ASCII del carattere. Questo permette di stamparlo come lettera o come numero, e persino di fare aritmetica su di esso.

char come intero

```
char lettera = 'A';
printf("%c", lettera); // → A
printf("%d", lettera); // → 65
// Aritmetica ASCII:
'A' + 1  → 'B'  (codice 66)
'A' + 2  → 'C'  (codice 67)
```

Sequenze di escape

`\n`	a capo (10)
`\t`	tabulazione (9)
`\\`	barra rovescia
`\'`	apice singolo
`\0`	carattere null (0)

Simulazione del booleano con `int`

```
int maggiorenne = (eta >= 18); // vale 0 o 1
int valido = (eta >= 0) && (eta <= 120); // vale 0 o 1
```

Costanti: `#define` vs `const`

```
#define PI_GRECO 3.14159   // preprocessore: sostituzione testuale, senza tipo
const double PI = 3.14159; // variabile tipizzata, protetta da modifiche
```

Caratteristica	`#define`	`const`
Tipo	nessuno	sì (double, int, …)
Visibile al debugger	no	sì
Uso in `#if`	sì	no
Errore di sintassi	a runtime	a compile-time

Convenzione nomi

I nomi `#define` si scrivono in `MAIUSCOLO_CON_UNDERSCORE` per distinguerli dalle variabili.

Quando usare `const`

Preferire `const` quando il tipo è importante per la correttezza dei calcoli e si vuole il supporto del debugger.

printf: specificatori di formato

Ogni specificatore indica a printf come interpretare e stampare un valore. L'ordine degli argomenti deve corrispondere esattamente agli specificatori nella stringa di formato.

Spec.	Tipo	Esempio → Output
%d	int	printf("%d", 42) → 42
%u	unsigned int	printf("%u", 300u) → 300
%ld	long	printf("%ld", 1L) → 1
%f	double (6 dec.)	printf("%f", 3.14) → 3.140000
%e	notaz. scientifica	printf("%e", 3.14) → 3.140000e+00
%g	automatico	printf("%g", 3.14) → 3.14
%c	char	printf("%c", 'A') → A
%%	simbolo %	printf("100%%") → 100%

Le sequenze di escape più usate: \n (a capo), \t (tab), \\ (barra rovescia), \" (virgolette doppie).

printf: formattazione avanzata

Sintassi completa: `%[flag][larghezza][.precisione]tipo`

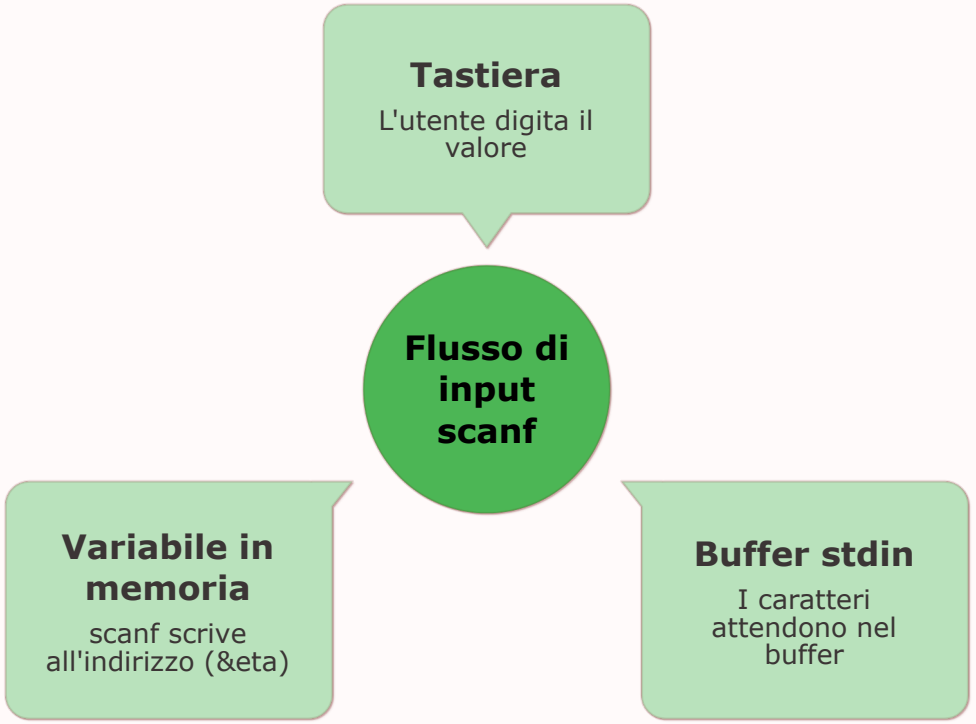
Formato	Valore	Output	Spiegazione
`%5d`	42	`42`	padding a sinistra (5 caratteri totali)
`%-5d`	42	`42`	allineato a sinistra
`%05d`	42	00042	zeri iniziali al posto degli spazi
`%.2f`	3.14159	`3.14`	solo 2 cifre decimali
`%10.3f`	3.14159	`3.142`	larghezza 10, 3 decimali

Esempio di tabella prodotti allineata con formattazione avanzata:

```
printf("%-4d %-15s %8.2f %6d\n", 1, "Penna", 0.50, 200);
printf("%-4d %-15s %8.2f %6d\n", 2, "Quaderno", 2.90, 85);
// Output:
// 1 Penna 0.50 200
// 2 Quaderno 2.90 85
```

`scanf`: lettura da tastiera

`scanf` legge dal buffer di input (`stdin`) e salva i valori nelle variabili. L'operatore `&` ("indirizzo di") è obbligatorio: dice a `scanf` *dove* in memoria salvare il valore letto.



✅ Uso corretto

```
int eta;
scanf("%d", &eta);
double peso;
scanf("%lf", &peso);
char sesso;
scanf(" %c", &sesso);
```

❌ Errori comuni

```
scanf("%d", eta);   // manca &!
scanf("%f", &peso); // %f per double!
scanf("%c", &sesso);// legge \n residuo!
```

Lo spazio prima di `%c` salta il `\n` lasciato nel buffer dall'invio precedente.

Operatori aritmetici

L'operazione più insidiosa è la **divisione intera**: quando entrambi gli operandi sono `int`, il risultato è troncato. Basta un `double` per ottenere la divisione reale.

Operandi	Operazione	Risultato	Spiegazione
`17 / 5`	int / int	`3`	⚠️ parte decimale persa!
`17.0 / 5`	double / int	`3.4`	un double basta per il risultato reale
`(double)17 / 5`	cast + div	`3.4`	cast esplicito: preferibile con variabili

Operatore modulo `%` (solo interi)

17 % 5 = 2

17 = 5×3 + 2 → resto della divisione

10 % 2 = 0

Resto zero → numero pari

17 % 10 = 7

Estrae l'ultima cifra decimale

Precedenza: `*`, `/`, `%` vengono valutati prima di `+` e `-`. Le parentesi forzano l'ordine desiderato.

Operatori relazionali e logici

In C non esiste un tipo booleano nativo: gli operatori relazionali e logici producono sempre `0` **(falso) o** `1` **(vero)**, un normale `int` salvabile in una variabile.

```
int maggiorenne = (eta >= 18);            // 0 o 1
int in_range    = (eta >= 18) && (eta <= 65);// 0 o 1
int fuori_range = (eta < 0) || (eta > 120); // 0 o 1
int non_valido  = !in_range;              // 0 o 1
```

Corto circuito con `&&`

Se il primo operando è `0` (falso), il secondo **non viene valutato**. Utile per evitare divisioni per zero o accessi a puntatori nulli.

Corto circuito con `||`

Se il primo operando è `1` (vero), il secondo **non viene valutato**. Il risultato è già determinato dal primo operando.

Non servono `if/else` per calcolare un valore booleano: l'espressione relazionale è già un valore `0/1` direttamente assegnabile.

Operatori di assegnazione e incremento

Operatori composti

`x += 3`	equivale a `x = x + 3`
`x -= 3`	equivale a `x = x - 3`
`x *= 2`	equivale a `x = x * 2`
`x /= 4`	equivale a `x = x / 4`
`x %= 4`	equivale a `x = x % 4`

Prefisso vs Postfisso

```
a = 5; b = ++a;
// a=6, b=6
// prima incrementa, poi assegna

a = 5; b = a++;
// a=6, b=5
// prima assegna, poi incrementa
```

Uso tipico come accumulatore

```
somma += valore; // invece di somma = somma + valore
totale *= 1.22; // applica IVA al 22% in un solo passaggio
```

Operatore ternario e operatori bit a bit

Operatore ternario `?:`

È un'**espressione** (non un'istruzione) che restituisce un valore in base alla condizione:

```
condizione ? valore_se_vero : valore_se_falso

double prezzo =
  (fascia == 1) ? 0.28 :
  (fascia == 2) ? 0.22 : 0.18;
```

Operatori bit a bit

Su `a=60` (`0011 1100`) e `b=15` (`0000 1111`):

`a & b`	`= 12`	AND
`a | b`	`= 63`	OR
`a ^ b`	`= 51`	XOR
`a << 1`	`= 120`	×2
`a >> 1`	`= 30`	÷2

Errori comuni da evitare

1. Divisione intera involontaria

```
double r = 7 / 2;        // r = 3.0, NON 3.5!
double r = (double)7 / 2; // corretto
```

2. Confondere `=` con `==`

```
if (eta = 18) { ... }  // assegna 18, sempre vero!
if (eta == 18) { ... } // corretto: confronta
```

3. Manca `&` in `scanf`

```
scanf("%d", eta);   // comportamento indefinito!
scanf("%d", &eta);  // corretto
```

4. `%f` per `double` in `scanf`

```
scanf("%f", &peso);  // sbagliato: peso è double!
scanf("%lf", &peso); // corretto
```

5. Confronto tra reali con `==`

```
if (x == 0.3) { ... } // può non entrare mai!
if (fabs(x-0.3) < 1e-9) {} // corretto: tolleranza
```

Riepilogo concetti

Argomento	Concetto chiave	Da ricordare
Hello world	Struttura minima: include, main, return	Un solo `main` per programma
Tipi interi	`short`, `int`, `long`, `unsigned`	Scegliere il tipo giusto per l'intervallo
Tipi reali	`float`, `double`, `long double`	Non usare `==` per confrontare reali
`char`	Intero a 8 bit, codice ASCII	`%c` carattere, `%d` codice numerico
Costanti	`#define` (testo) vs `const` (tipizzato)	MAIUSCOLO per i nomi `#define`
`printf`	Specificatori + larghezza/precisione	`%lf` in scanf, `%f` o `%lf` in printf
`scanf`	`&` obbligatorio, spazio prima di `%c`	`%lf` per double, non `%f`
Aritmetici	`/` tra int tronca; `%` solo per interi	Cast `(double)` per divisione reale
Relaz./logici	Risultato sempre 0 o 1 (`int`)	`&&` e `||` con corto circuito
Assegnazione	`+=` `-=` `*=` `/=` `%=`; `++`/`--`	Prefisso incrementa prima
Ternario	`condizione ? vero : falso`	Espressione, non istruzione
Bit a bit	`&` `|` `^` `~` `<<` `>>`	`<<` ×2; `>>` ÷2

Esempi pratici: Hello World (concetti base)

1

010-hello-world.c

Obiettivo: scrivere il programma più semplice possibile e verificare che l'ambiente di sviluppo funzioni correttamente.

Tecnica: struttura minima con `#include <stdio.h>`, `int main(void)`, una `printf` con `\n`, `return 0`. Nessuna variabile dichiarata.

2

020-struttura-programma.c

Obiettivo: mostrare tutte le parti di un programma C con etichette e commenti esplicativi su ogni sezione.

Tecnica: direttive `#include` e `#define`; dichiarazione di variabili `int` e `double`; `printf` con `%d` e `%.2f`; commenti `/* */` su ogni sezione.

3

030-commenti.c

Obiettivo: illustrare le due sintassi di commento e le buone pratiche di documentazione del codice.

Tecnica: commento `/* */` multi-riga; commento `//` a singola riga; esempio di commento utile (spiega il perché) vs inutile (ripete il cosa).

Esempi pratici: Hello World (problemi)

1

040-saluto-personalizzato.c

Problema: chiedere giorno, mese, anno di nascita e anno corrente; calcolare l'età e stampare un saluto con riepilogo formattato.

Tecnica: quattro variabili `int`; `scanf("%d", &var)` per ogni dato; `eta = anno_corrente - anno_nascita`; formattazione data con `%02d`. Nessun array, nessuna stringa.

2

050-biglietto-da-visita.c

Problema: chiedere dati numerici e stamparli dentro un biglietto da visita con bordo ASCII.

Tecnica: testo fisso nelle `printf`; `%03d` per il prefisso; `%06d` per il codice; costante `#define LARGHEZZA` per la dimensione del bordo.

3

060-intestazione-documento.c

Problema: chiedere anno scolastico, classe, sezione, numero documento e data; produrre un'intestazione scolastica formattata.

Tecnica: `anno_fine = anno_inizio + 1` (calcolato, non inserito); `%03d` per il numero documento; `%02d` per giorno e mese; sezione come intero.

Esempi pratici: Variabili e Tipi (concetti base)

1

010-tipi-interi.c

Obiettivo: mostrare dimensioni, limiti e valori delle principali variabili intere.

Tecnica: `sizeof(tipo)` con `%zu`; costanti di `<limits.h>` (`INT_MAX`, `SHRT_MIN`…); suffissi `L`, `U`, `LL`; overflow mostrato nei commenti.

2

020-tipi-reali.c

Obiettivo: confrontare la precisione di `float`, `double` e `long double` su un caso concreto.

Tecnica: `FLT_DIG`, `DBL_DIG` da `<float.h>`; stampa con `%.20f`; notazione scientifica con `1.496e8`; dimostrazione del pericolo di `==` tra reali.

3

030-caratteri-e-booleani.c

Obiettivo: mostrare che `char` è un intero e come si simula il booleano senza `<stdbool.h>`.

Tecnica: stampa con `%c` e `%d` sullo stesso `char`; aritmetica ASCII; sequenze di escape; espressione `(eta >= 18)` assegnata a `int maggiorenne`.

4

040-costanti.c

Obiettivo: confrontare `#define` e `const` in esempi pratici.

Tecnica: costanti matematiche (`PI_GRECO`, `GRAVITA`) e applicative (`ETA_MINIMA`, `IVA`); tentativo di modifica mostrato come errore di compilazione.

Esempi pratici: Variabili e Tipi (problemi)

1

050-conversione-temperature.c

Problema: leggere una temperatura in Celsius e stamparla convertita in Fahrenheit e Kelvin.

Tecnica: `#define FATTORE_F (9.0/5.0)` e `OFFSET_K 273.15`; divisione `9.0/5.0` reale (non `9/5` intera!); `%lf` in `scanf` e `%.2f` in `printf`.

2

060-calcolo-area.c

Problema: scegliere una figura (1=quadrato, 2=rettangolo, 3=cerchio) e calcolarne area e perimetro.

Tecnica: `if/else if` per la selezione; costante `PI_GRECO` con `#define`; `scanf("%lf", &raggio);` `return 1` per scelta non valida.

3

070-scheda-prodotto.c

Problema: inserire codice, prezzo, quantità e sconto; calcolare prezzo scontato e valore totale di magazzino.

Tecnica: tipi misti `int` e `double`; `fattore_sconto = sconto / 100.0`; cast `(double)quantita` prima della moltiplicazione; `%06d` per il codice.

Esempi pratici: Input/Output (concetti base)

1

010-printf-basi.c

Obiettivo: mostrare sistematicamente tutti i principali specificatori di formato con esempi concreti.

Tecnica: `%d`, `%u`, `%ld`, `%f`, `%e`, `%c`, `%%`; sequenze `\n`, `\t`, `\\`, `\"`; più valori nella stessa `printf` con ordine degli argomenti corrispondente.

2

020-scanf-basi.c

Obiettivo: leggere variabili di tipo diverso dalla tastiera e riepilogarle a schermo.

Tecnica: `&` come "indirizzo dove salvare"; `%lf` obbligatorio per `double`; spazio prima di `" %c"` per saltare il `\n` residuo; errori comuni mostrati in commento.

3

030-formattazione-avanzata.c

Obiettivo: costruire output tabellare allineato usando le opzioni di formato di `printf`.

Tecnica: `%5d` padding, `%-5d` allineamento sinistro, `%05d` zeri, `%.2f` precisione, `%10.3f` combinazione; tabella prodotti con quattro colonne allineate.

Esempi pratici: Input/Output (problemi)

1

040-scheda-studente.c

Problema: inserire matricola, anno di corso, tre voti e voto di laurea; calcolare la media e stampare una scheda formattata.

Tecnica: `%07d` per la matricola; `media = (v1+v2+v3)/3.0`; `scanf` con tipi misti; larghezze di campo per allineare le colonne.

2

050-scontrino.c

Problema: inserire prezzo e quantità di tre articoli; calcolare subtotali, totale imponibile, IVA al 22% e totale ivato; stampare uno scontrino con bordo ASCII.

Tecnica: cast `(double)quantita`; `#define IVA 0.22`; bordo `+---+` con colonne `%-18s`, `%4d`, `%8.2f`.

3

060-dati-veicolo.c

Problema: inserire dati di un veicolo e stampare una scheda tecnica con anzianità e conversione potenza kW→CV.

Tecnica: `#define ANNO_CORRENTE 2024` e `KW_IN_CV 1.35962`; `anzianita = ANNO_CORRENTE - anno_immatricolazione`; scheda con separatori e colonne `%8.1f`.

Esempi pratici: Operatori (concetti base)

1

010-operatori-aritmetici.c

Obiettivo: mostrare il comportamento di tutti gli operatori aritmetici, con enfasi sulla divisione intera.

Tecnica: confronto diretto `17/5=3` vs `17.0/5=3.4`; cast esplicito; modulo `%` con applicazioni pratiche (parità, ultima cifra); precedenza con e senza parentesi.

2

020-operatori-relazionali-e-logici.c

Obiettivo: dimostrare che gli operatori relazionali e logici producono valori interi 0/1 salvabili in variabili.

Tecnica: `int maggiorenne = (eta >= 18)`; condizioni composte con `&&` e `||`; operatore `!`; corto circuito spiegato nei commenti; nessun `if/else`.

3

030-operatori-di-assegnazione.c

Obiettivo: confrontare le forme compatte con quelle estese e mostrare la differenza tra prefisso e postfisso.

Tecnica: `+=`, `-=`, `*=`, `/=`, `%=` in sequenza; `b = ++a` vs `b = a++`; pericolo del doppio incremento nella stessa espressione.

4

040-operatori-bit-a-bit.c

Obiettivo: visualizzare l'effetto di ogni operatore bit a bit con rappresentazione binaria nei commenti.

Tecnica: `a=60` (`0011 1100`) e `b=15` (`0000 1111`); schema binario per `&`, `|`, `^`; shift come ×2 e ÷2; test/set/clear di un bit con maschera `1u << 2`.

Esempi pratici: Operatori (problemi)

1

050-calcolo-bolletta.c

Problema: calcolare la bolletta elettrica da letture del contatore, potenza impegnata e fascia oraria.

Tecnica: prezzo per fascia con ternario annidato `(fascia==1)?F1:(fascia==2)?F2:F3`; `kwh = lettura_attuale - lettura_precedente`; IVA domestica al 10%; zero `if/else`.

2

060-indice-massa-corporea.c

Problema: calcolare l'IMC e determinare la categoria (sottopeso / normopeso / sovrappeso / obesità).

Tecnica: `imc = peso / (altezza * altezza)`; quattro flag `int sottopeso = (imc < 18.5)`; categoria con ternario annidato su `%s`; zero `if/else`.

3

070-cambio-valuta.c

Problema: convertire un importo in EUR nella valuta scelta (USD, GBP, JPY, CHF) sottraendo la commissione.

Tecnica: tasso selezionato con ternario annidato su `scelta`; `importo_commissione *= COMMISSIONE`; simbolo valuta con secondo ternario in `printf`; zero `if/else`.